

EPPS 6302 Methods of Data Collection and Production:

Assignment 2

Google Trends: two ways to get the same data

Instructor: Karl Ho

Fall 2026

What you will produce

The same Google Trends series pulled two ways — once by hand from the website, once from R with **gtrendsR** — and a short note on what differs between them and why.

A. Google Trends data

1. Collect it from the website

- Go to <https://trends.google.com> and search **Trump**, **Kamala Harris** and **Election**.
- Download the data as CSV.
- Record the query.** Before you close the tab, write down every choice you made — search terms, geography, time window, category, search type (web / news / images / YouTube), and the **date and time you pulled it**. Without these, nobody — including you — can reproduce this file.
- Repeat once with a term of your own that is currently in the news.

2. Look at what you got

- What range of **dates** does the file cover?
- What is the **interval** — hourly, daily, weekly, monthly? Now change the date window and download again. Did the interval change?
- What are the **units**? What does a value of 100 mean, and 0?
- Now remove one search term and download again. Did the numbers for the remaining terms stay the same?

Trends is not a count. Google divides each point by total searches in that geography and time range, then scales the result 0–100, where 100 is the peak for *that query*. It is computed from a *sample* of searches. So the numbers are a share, rescaled — not volume — and they are only comparable inside a single query. See support.google.com/trends/answer/4365533.

3. Collect it from R

- Use the **gtrendsR** package and the `gtrendsR01.R` program. The first call in that script is the same query you just ran by hand:

```
library(gtrendsR)
library(readr)

TrumpHarrisElection <- gtrends(c("Trump", "Harris", "election"),
                              onlyInterest = TRUE,
                              geo          = "US",
                              gprop       = "web",
                              time        = "today+5-y", # last five years
                              category    = 0)

the_df <- TrumpHarrisElection$interest_over_time
plot(TrumpHarrisElection)

# Cache the raw pull - the next call will not return the same numbers
write_csv(the_df, paste0("gtrends_us_5y_", Sys.Date(), ".csv"))
```

- Work through the rest of the script: one term over its full history (`time = "all"`), the same term in another country, one term across `c("US", "GB", "TW")`, and three terms worldwide.
- Section 6 of the script runs `names(tct)`. That call left `onlyInterest` at its default, so the object carries several data frames. **Which of them did the website CSV not give you?**

A note on terms

The website instructions above say **Kamala Harris**. The script searches **Harris**.

- Pull both and compare the series. Are they the same?
- Harris** is also a county, a supermarket chain, and several thousand other people. What does that gap tell you about the difference between a search **term** and the **concept** you meant to measure?

4. Compare the two methods

- Line the two series up. **Are the numbers identical?** If not, by how much, and in which direction?
- What are the differences between the two methods? Consider at least:
 - Reproducibility — could someone else regenerate your file exactly?
 - Provenance — which method records the query parameters for you?
 - Scale — what happens when you need fifty terms instead of three?
 - Fragility — what breaks, and how would you know?
- Run section 7 of the script: the **same** call twice, a few minutes apart. Do you get the same numbers? What does your answer imply about every finding you might report from this source?

B. Write it up

- One page, in Quarto, with your code and at least one plot.
- Open with the query parameters from A.1.c — that block is a small **datasheet** for your data.
- Publish as `assign02.qmd` under **Assignments** on your website, and send the URL to the teaching assistant.

Be kind to the endpoint. Trends has no official public API; `gtrendsR` scrapes the same service your browser uses. Expect rate limiting — cache your first successful pull and work from the file.